



CMP5367: Artificial Intelligence and Machine Learning

Lectures 6: Naïve Bayes and Model Evaluation

Instructor: **Dr. Hadeel Saadany**

October 27th 2025

Outline

- Bayesian Classification
- Bayesian Theorem
- Naïve Bayesian Classification
- Model Evaluation
- Assessment details.

Bayesian Classification

Bayesian Classification

- Bayesian classification uses Bayes theorem to predict the occurrence of any event.
- Naive Bayes models extremely fast and simple classification algorithms.
- Often suitable for very high-dimensional datasets.
- Bayes theorem came into existence after Thomas Bayes.
- The method uses Bayes' theorem to calculate the conditional probability of each class given the observed features, and then selects the class with the highest probability as the predicted class for the data point.

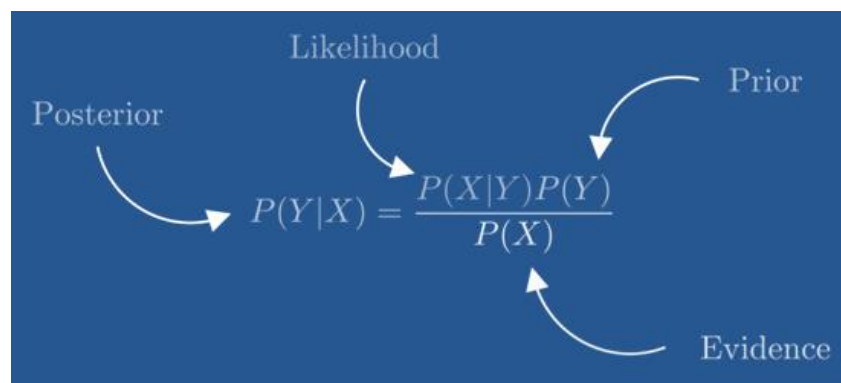
$$P(Y|X) = \frac{P(X \text{ and } Y)}{P(X)}$$

The conditional probabilities represent the probability of an event X given some other event Y has occurred

Bayes theorem

Bayes' Rule, allows us to “invert” conditional probabilities.

As a reminder, conditional probabilities represent the probability of an event given some other event has occurred



Y: Class (**spam or ham**).

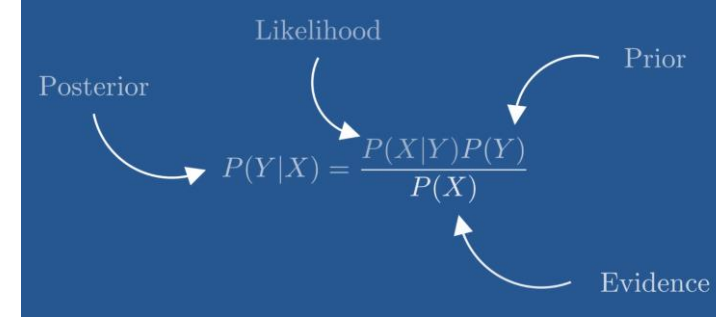
X: Feature (e.g. frequency of the word **money** in each mail category).

P(Y): the probability of having this class (**spam/dataset** or **ham/dataset**).

P(X): The probability of the feature X (word **money** in either **spam** or **ham**)

P(Y|X): the probability of having a **spam** email given we have X (**'money'**).

Bayesian Classification

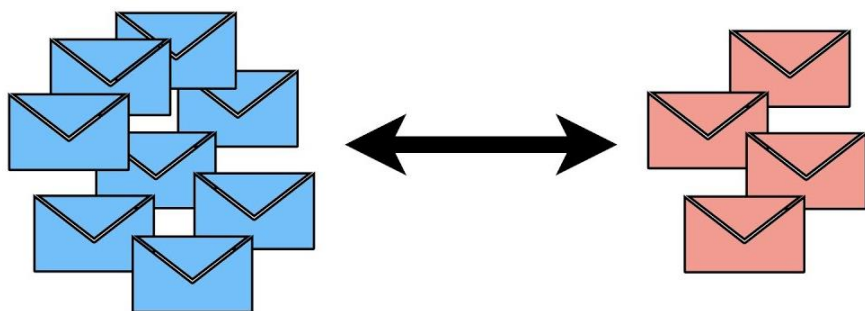


Here's how the process works:

- **Calculate Prior Probabilities:** Compute the prior probabilities of each class based on the training data. This represents the probability of each class occurring without considering any features.
- **Calculate Likelihoods:** Estimate the likelihood of observing each feature given each class. This represents how likely each feature is to occur given a specific class.
- **Apply Bayes' Theorem:** Use Bayes' theorem to calculate the **posterior probability** of each class given the observed features. This involves multiplying the prior probability of each class by the likelihood of observing the features given that class.
- **Make Predictions:** Assign the instance in the dataset to the class with the **highest posterior probability**.

Example: Detect Spam Emails

Dataset (8 normal and 4 spam)



$$\hat{P}(\text{spam}) = \frac{\text{\# of spam messages in training set}}{\text{\# of all messages in training set}}$$

Prior probabilities

We have 12 emails

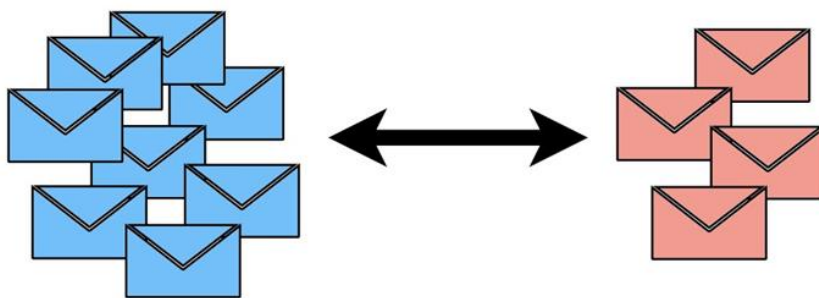
Prior probability for each class $P(Y)$ (Before I consider any features):

Normal = $8/12 = 0.66$

Spam = $4/12 = 0.33$

Prior probability acts as a weight to the conditional probability

Naïve Bayes Classification



Word	Frequency in Spam	Frequency in Ham (Normal email)	Likelihood in spam $P(X Y)$	Likelihood in Normal $P(X Y)$	$P(X)$ Frequency of the word/total words
Dear	2	8	$2/7 = 0.29$	$8/17 = 0.47$	$10/24$
Friend	1	5	$1/7 = 0.14$	$5/17 = 0.29$	$6/24$
Lunch	0	3	$0/7 = 0.0$	$3/17 = 0.18$	$3/24$
Money	4	1	$4/7 = 0.57$	$1/17 = 0.06$	$5/24$
Total number of words	7	17			

Naïve Bayes Classification

If a message has the words “Friend”, is it likely to be Spam or Normal?

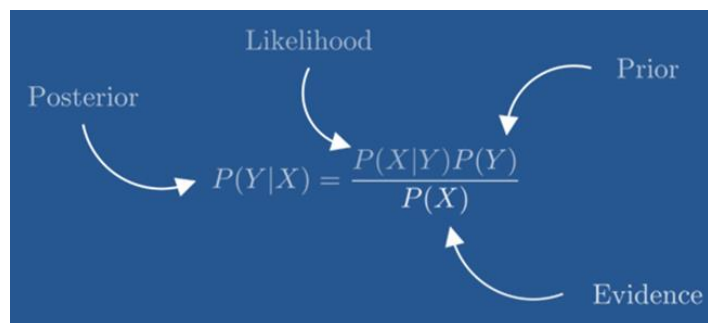
Prior probability for each class $P(Y)$:

Normal = $8/12 = 0.66$

Spam = $4/12 = 0.33$

$P(X)$

$P(\text{'Friend'}) = 6/24 = 0.25$



Likelihoods $P(X|Y)$

$p(\text{Dear} | \text{N}) = 0.47$

$p(\text{Friend} | \text{N}) = 0.29$

$p(\text{Lunch} | \text{N}) = 0.18$

$p(\text{Money} | \text{N}) = 0.06$

Posterior probability (Normal) = $0.29 \times 0.66 / 0.25 = 0.76$

Posterior probability (Spam) = $0.14 \times 0.33 / 0.25 = 0.18$

Dear Friend is likely to
be in the Normal Email

$p(\text{Dear} | \text{S}) = 0.29$

$p(\text{Friend} | \text{S}) = 0.14$

$p(\text{Lunch} | \text{S}) = 0.00$

$p(\text{Money} | \text{S}) = 0.57$

Quiz

If a message has the words “Money”, is it likely to be Spam or Normal?

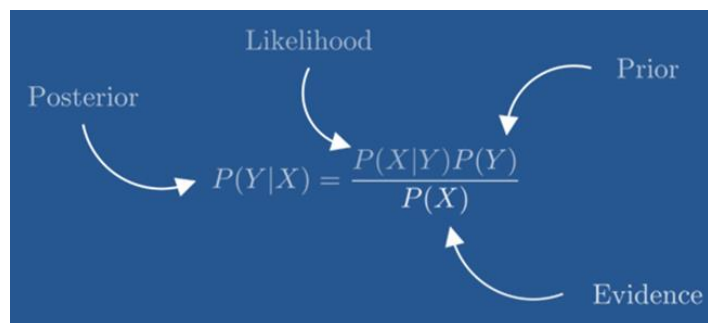
Prior probability for each class $P(Y)$:

Normal = $8/12 = 0.66$

Spam = $4/12 = 0.33$

$P(X)$

$P(\text{'Money'}) = 5/24 = 0.21$



Likelihoods $P(X|Y)$

$p(\text{Dear} | \text{N}) = 0.47$

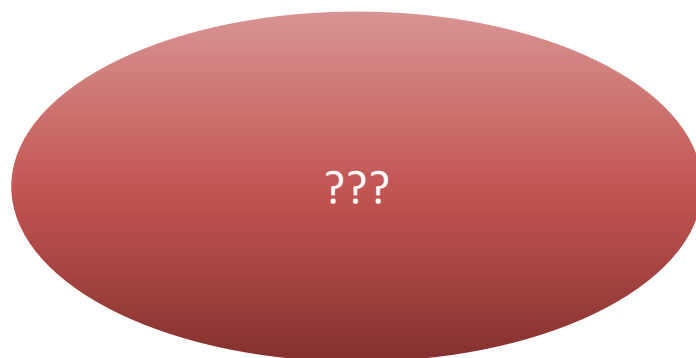
$p(\text{Friend} | \text{N}) = 0.29$

$p(\text{Lunch} | \text{N}) = 0.18$

$p(\text{Money} | \text{N}) = 0.06$

Posterior probability (Normal) = $.06 \times 0.66 / 0.21 = 0.19$

Posterior probability (Spam) = $0.57 \times 0.33 / 0.21 = 0.89$



$p(\text{Dear} | \text{S}) = 0.29$

$p(\text{Friend} | \text{S}) = 0.14$

$p(\text{Lunch} | \text{S}) = 0.00$

$p(\text{Money} | \text{S}) = 0.57$

Why is it Naïve?

- It assumes there is no relation between features (features are independent). We do not calculate this ($P(Y|X_1, X_2, X_3)$)
- In our example, it assumes no relation between words in the email

Pros:

- It is easy and fast to predict class.
- The classifier performs better compared to other machine learning models.
- It perform well in case of categorical input variables compared to numerical variable(s).

Cons:

- If categorical variable has a category (in test set), which was not observed in training data set, then model will assign a 0 (zero) probability and will be unable to make a prediction. This is often known as “Zero Frequency”. To solve this, we can use the smoothing technique (alpha).
- Another limitation of this algorithm is the assumption of independent predictors.

Naïve Bayes Classifier Applications

Naive Bayes classifiers are widely used in various applications, including text classification, email spam filtering, sentiment analysis, and recommendation systems.

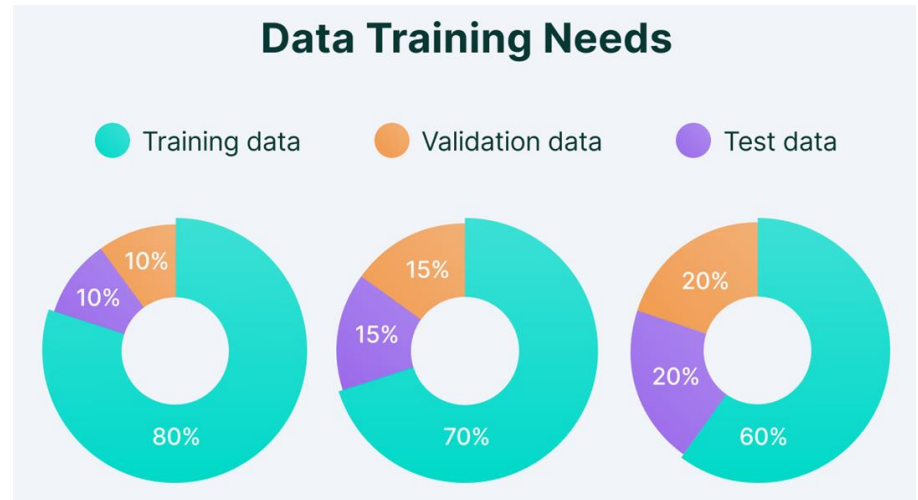
- Face recognition
- Credit scoring
- News classification
- Medical data classification
- Text classification such as Spam filtering and Sentiment analysis.
- And many more areas

Model Evaluation

Method 1: Model Training

Method 1: Training Model

- The dataset is divided into two subsets, use one for the training and the other for testing (in large datasets) (train-test-split).
- Common practice is to **train** the classifiers using **2 thirds** (including a slice for validation) of the dataset and **test** it using the **unseen third**. (66%, 34%)
- Validation data to **tune hyperparameters** and select the best model, and testing data to provide a final, unbiased evaluation of the model's performance on unseen data.



Method 2 Training Model: Cross Validation

- Cross-validation is a technique used in machine learning to assess the performance of a predictive model.
- It helps to evaluate how well a model will generalize to unseen data.
- The basic idea behind cross-validation is to partition the dataset into multiple subsets, called folds, and then train the model on several combinations of these folds.
- - Aggregate the results (e.g., average accuracy, F1-score) across all folds to get a final performance estimate

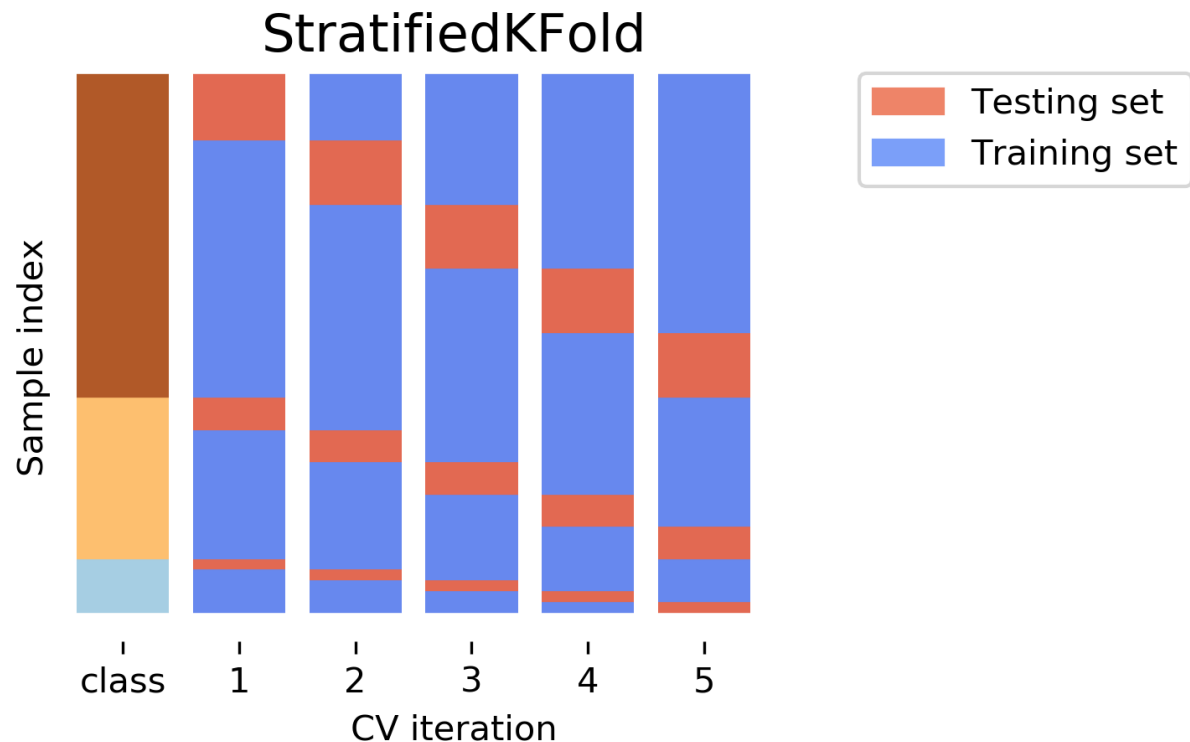
Advantages of Cross-Validation:

- Better Performance Estimate
- Reduces Variance
- Maximizes Data Usage



Method3 Training: Stratified k-Fold Cross Validation

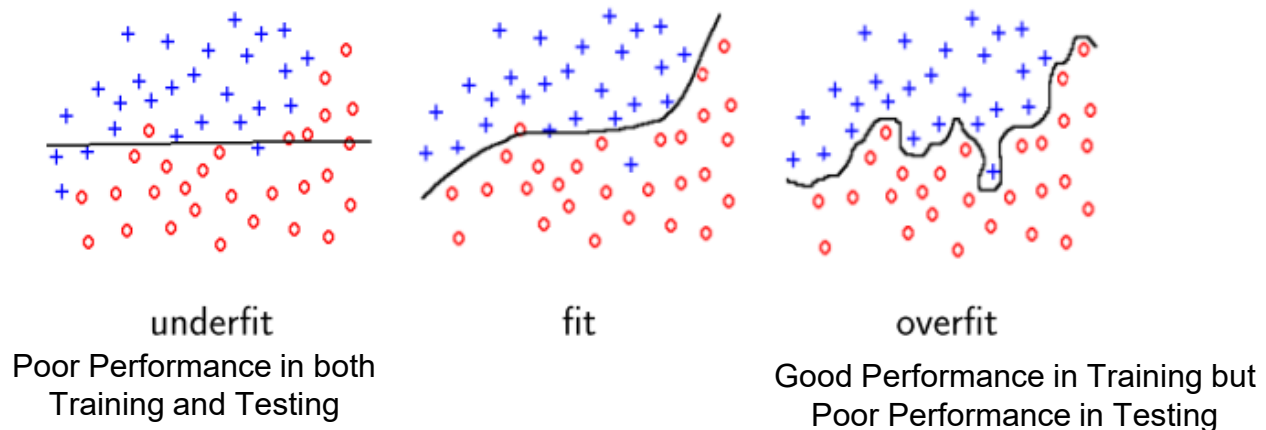
- Stratified k-Fold Cross-Validation: each fold has approximately the same percentage of samples of each target class as the full dataset.
- Ensures that **each fold contains a proportional representation of the classes present in the dataset**, helpful for imbalanced datasets.
- **Use one fold as the validation set.** Aggregate performance metrics across all folds.



What is Overfitting and Underfitting?

- **Overfitting:** Overfitting occurs when a model learns to capture noise and random fluctuations in the training data rather than the essential patterns. As a result, the model performs well on the training data but fails to generalize to unseen/ testing data.
- **Underfitting:** Underfitting occurs when a model is too simple to capture the underlying structure of the data, resulting in poor performance on both the training and validation/ testing datasets.

Cross-validation, regularization techniques and careful model selection are important strategies for mitigating overfitting and underfitting in practice.



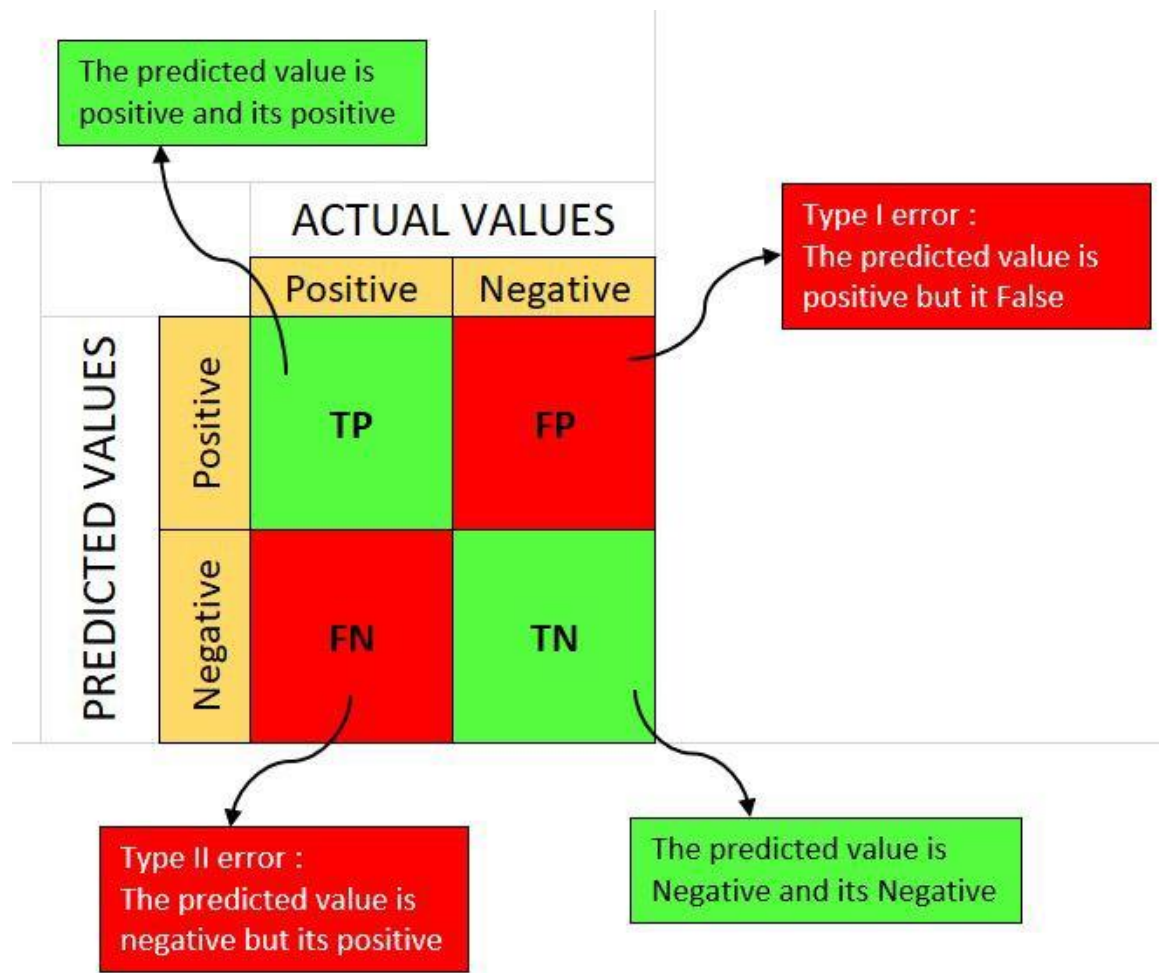
Evaluation metrics

- **Accuracy:**

$$\text{Accuracy} = \frac{\text{number of correctly classified instances}}{\text{total number of instances}} \times 100$$

- It assumes equal cost for all classes
- Misleading in unbalanced datasets
- It doesn't differentiate between different types of errors
- Ex 1:
 - Cancer Dataset: 10000 instances, 9990 are **normal**, 10 are **ill** , If our model classified all instances as **normal** accuracy will be 99.9 %
 - Medical diagnosis: 95 % healthy, 5% disease.

Binary Classification Confusion Matrix



Recall and Precision

Recall:

$$\text{Recall} = \frac{TP}{TP + FN}$$

The model's ability to capture the positive class in all the positives I have

Precision:

$$\text{Precision} = \frac{TP}{TP + FP}$$

The model's ability to capture the positive class in all my data

F-measure:

$$F\ measure = \frac{1}{\frac{1}{\text{Recall}} + \frac{1}{\text{Precision}}} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times TP}{2 \times TP + FP + FN}$$

Example for Precision and Recall

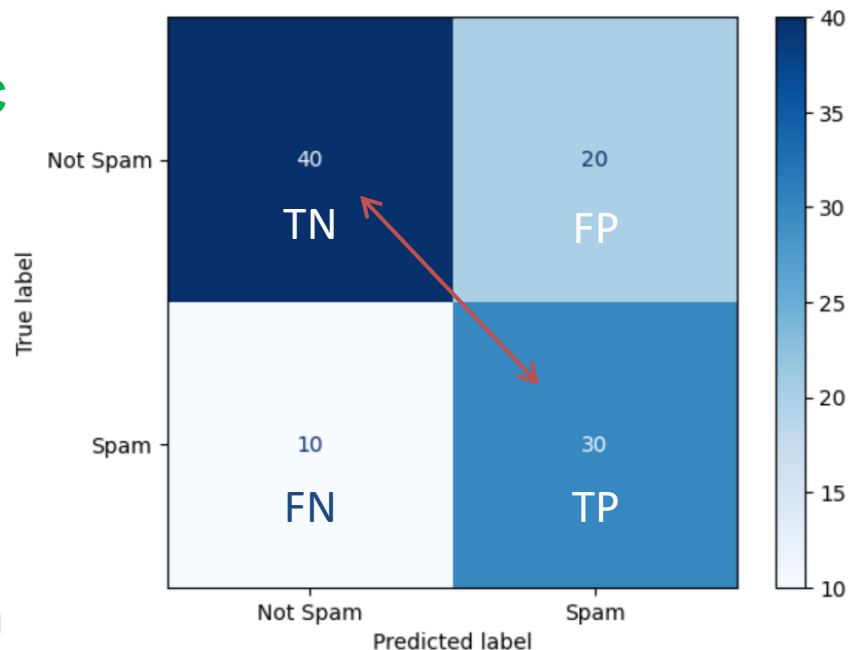
Imagine you build a model to classify emails as **spam** or **not spam**.

You test it on 100 emails:

- **Actual spam emails:** 40 (**Positive Class**)
- **Actual non-spam emails:** 60 (**Negative C**

Your model predicts:

- **Spam:** 50 emails
 - Of these, 30 are actually spam (**True Positives**)
 - 20 are not spam (**False Positives**)
- **Not spam:** 50 emails
 - Of these, 10 are actually spam (**False Negatives**)
 - 40 are correctly identified as not spam (**Negatives**)
- **Precision** = $TP / (TP + FP) = 30 / (30 + 20) = 0.60$
→ **60% of emails predicted as spam were actually spam.**
- **Recall** = $TP / (TP + FN) = 30 / (30 + 10) = 0.75$
→ **75% of actual spam emails were correctly identified.**



Good
Predictions

Interpretation of Confusion Matrix (Important)

- A **high precision** means **fewer false alarms** (non-spam marked as spam).
- A **high recall** means **fewer missed spam emails**.

*If you're building a spam filter, you might prioritize **recall** to catch as much spam as possible, even if a few legitimate emails are misclassified.*

- **Medical diagnosis:** Precision ensures that diagnosed patients truly have the disease; recall ensures most sick patients are identified.
- **Search engines:** Precision means relevant results are shown; recall means most relevant documents are retrieved.

Can you think of examples where Recall is more important than precision???

Questions?

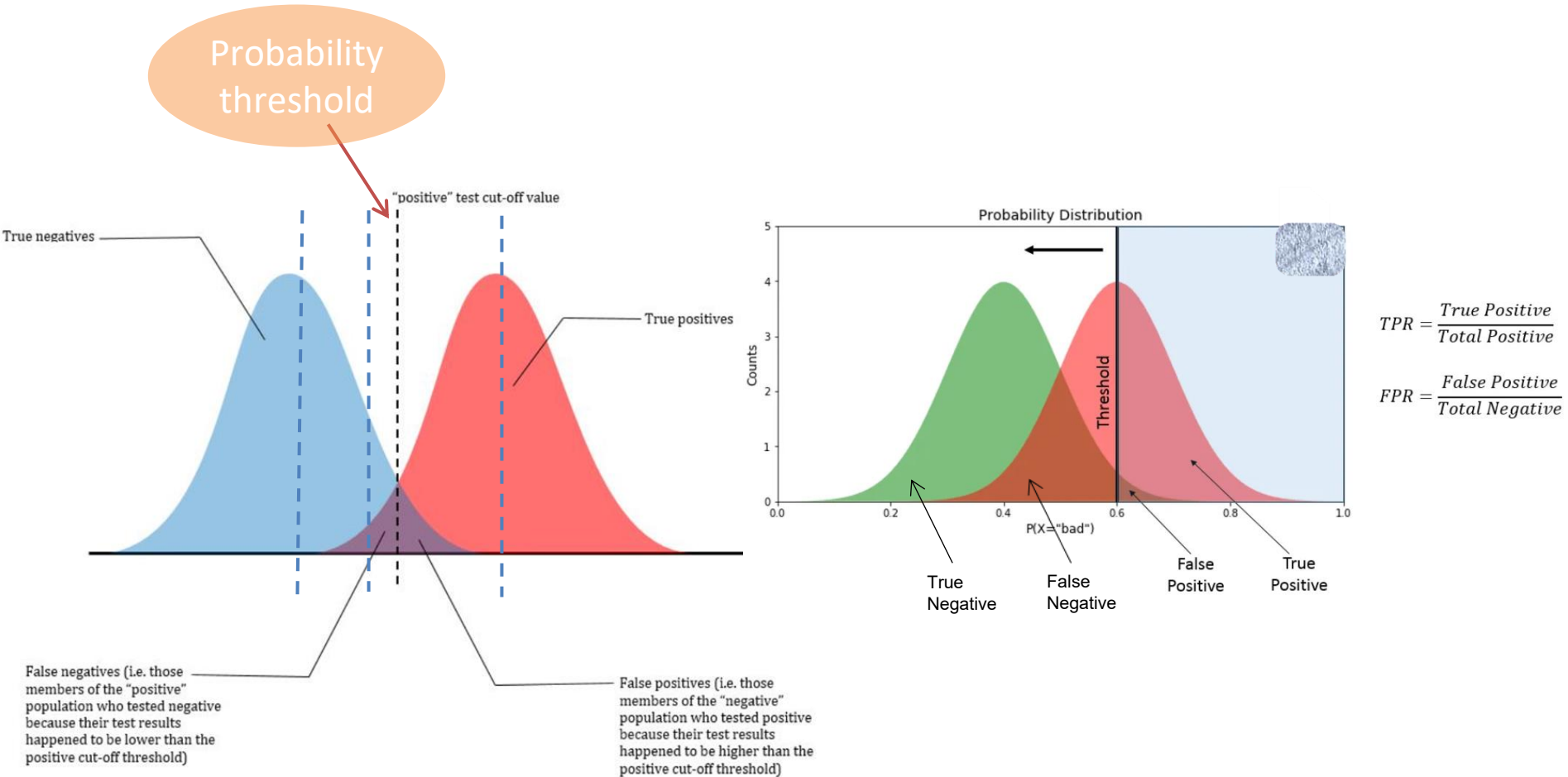
Quiz



ROC Curve

- ❑ Receiver Operating Characteristic (ROC) curve is a graphical representation that illustrates the performance of a binary classification model across different threshold settings.
- ❑ It basically plots the **True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings.**
- ❑ It's a widely used tool for evaluating the performance of classification algorithms.

ROC for a disease test: Positive Covid or Negative?



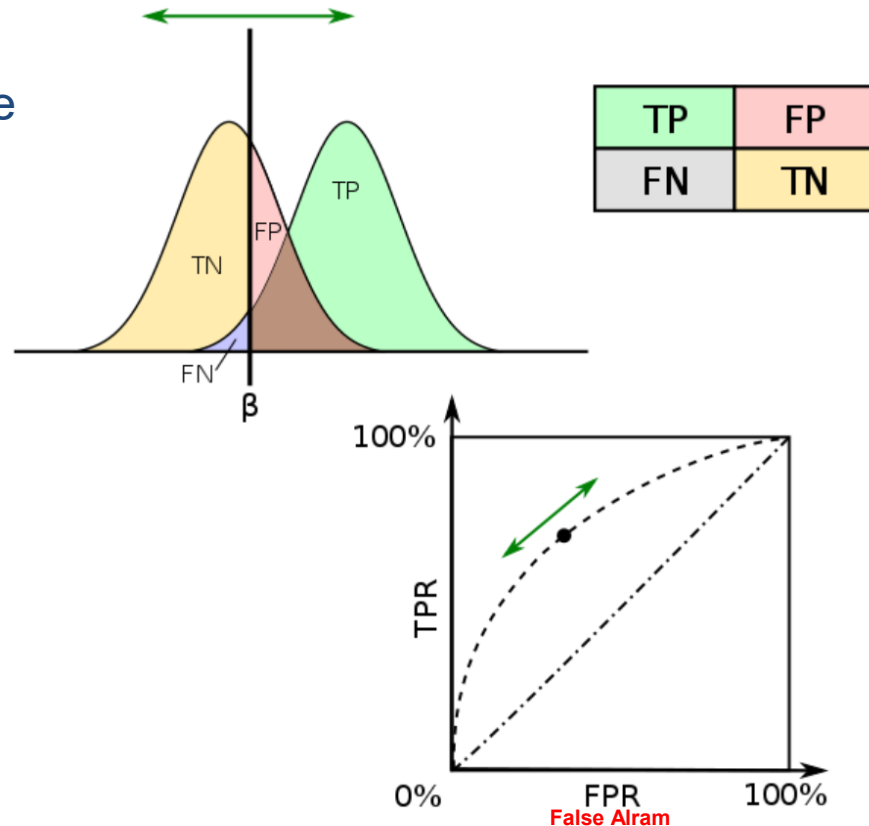
- At a low threshold, it flags almost everyone → high recall, but many false positives.
- At a high threshold, it flags only the most confident cases → low recall, but fewer false positives.
- ROC curve shows this trade-off visually.

How do we plot ROC curve?

How do we define or plot the ROC Curve?

ROC Curve is plotted by varying the thresholds and recording the classifier's performance at each threshold. If I change the threshold, the TPR and FPR is going to change

The ROC graph summarizes all the possible confusion matrices that each threshold produces



ROC Curve plotted when threshold β is varied. Source: Wikimedia Commons 2015. *

ROC curve

This curve decides on the best probability threshold for my model based on my use-case. Which scenario do I need to prioritise :

1. Lower priority to false positive (e.g. predicting all patients who have positive COVID even if I predict some as false positive, i.e. patients who do not have COVID but get further tests). **I need a model that does not make any false negatives.**
2. Lower priority to false negative (e.g. a spam email is not deleted as it is considered Normal). **I need a model that does not make false positives and delete my good emails.**

ROC Curve (cont.)

- **ROC Curve:**

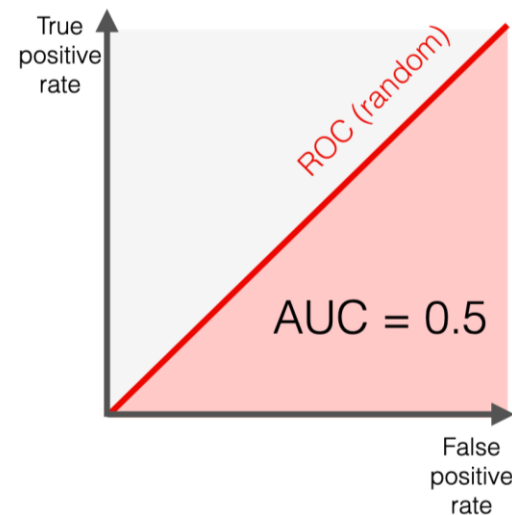
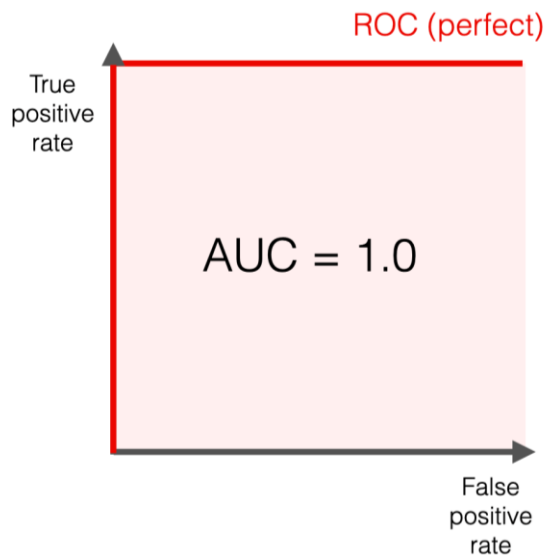
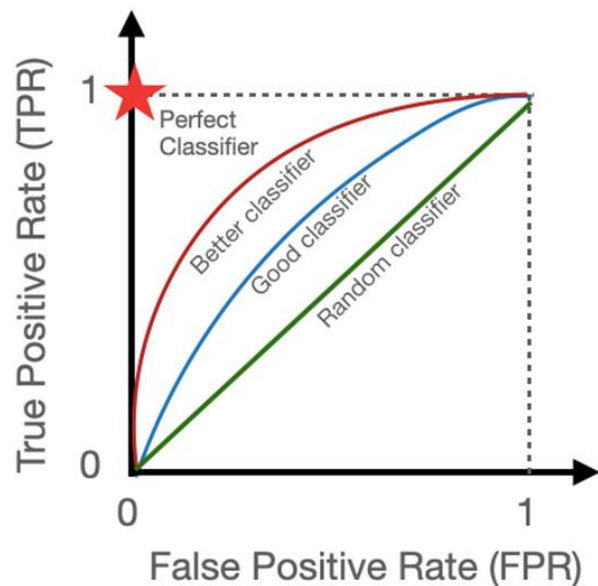
- The ROC curve is created by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR) for different threshold values.
- Each point on the ROC curve represents a specific threshold setting.
- The diagonal line (from (0,0) to (1,1)) represents random guessing, where the model's performance is no better than chance.



ROC Curve (cont.)

- **Area Under the Curve (AUC):**

- The **area under the ROC curve (AUC)** is a single metric that summarizes the overall performance of the classification model.
- AUC ranges from 0 to 1, where a higher AUC value indicates better discrimination ability of the model.
- **AUC = 1.0** → perfect classification
- **AUC = 0.5** → random guessing
- **AUC < 0.5** → worse than random (model may be inverted)



What we learned today?

- Bayesian Classification
 - Bayesian Theorem
 - Naïve Bayesian classification
 - Model Evaluation
-
- Assessment 1 Details